

# Algoritmos

Calculito

2026-04-28

## Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

## Probando primeros comandos de R

R es un lenguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

## Uso de data sets o base de datos internos de R

Usando el comando `data()` en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos están cargados en tablas de las cuales podemos elegir columnas específicas si escribimos el signo `$` después del comando de información específico que queremos.

También está el comando `summary()`, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed
```

```
# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

```
summary(cars)
```

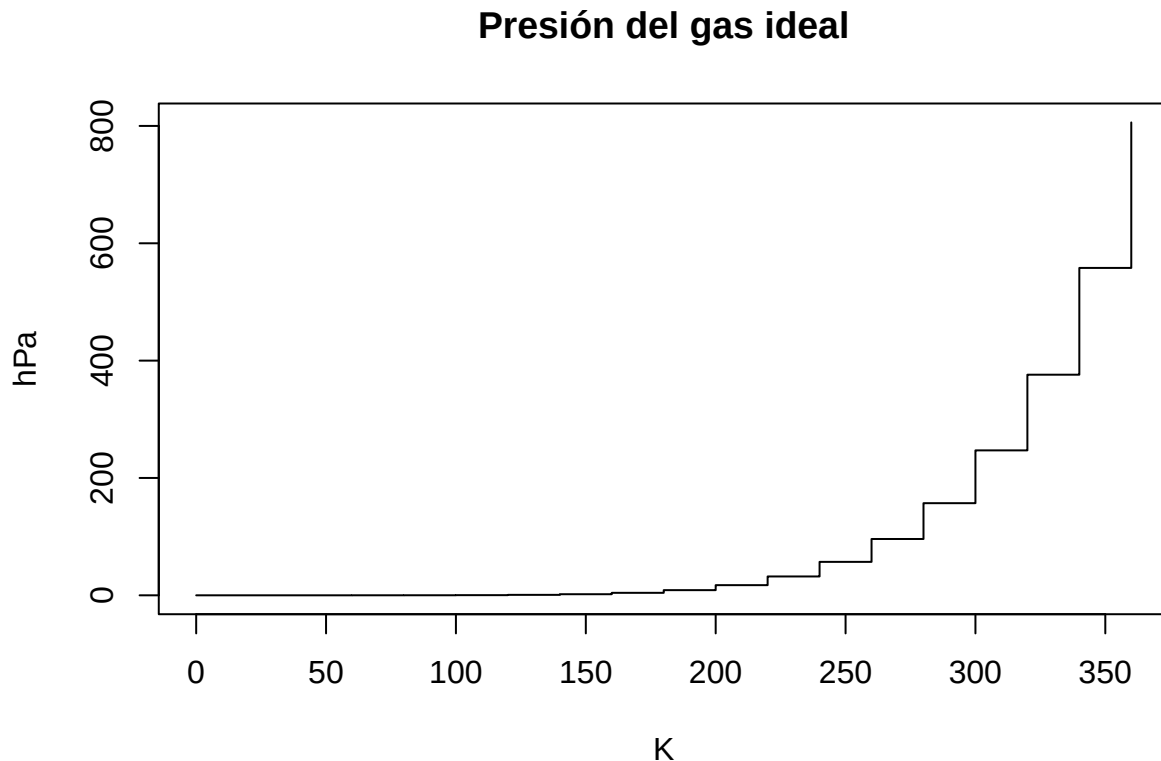
```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean    : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
```

```
## Max. :25.0 Max. :120.00
```

## Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahí nos va a explicar cada comando

```
plot(pressure, type = "s",  
     main = "Presión del gas ideal",  
     ylab = "hPa",  
     xlab = "K")
```



## Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el “error experimental”, en este caso, 5).

```
z1 <- rnorm(350, 22, 5)  
z1
```

```
## [1] 13.360937 27.257389 22.376123 20.648116 19.137378 28.455696 20.547006  
## [8] 16.116943 23.431790 18.884579 22.511272 21.098571 18.276283 24.388998  
## [15] 27.626834 22.980671 25.347106 21.606474 17.670424 16.892109 27.630783  
## [22] 26.030737 25.980968 19.625679 27.403371 14.246872 26.474102 19.419074  
## [29] 26.433933 25.005493 21.601759 19.664995 25.456304 22.696903 18.586038  
## [36] 22.762399 24.112536 24.120661 17.299228 22.612453 27.478800 25.746059
```

```
## [43] 15.243662 17.247141 13.816272 24.561123 17.526823 28.422633 23.563985
## [50] 25.486412 26.357607 24.373787 29.982621 10.042273 19.980255 22.313143
## [57] 28.776370 24.139306 25.743167 27.526848 17.422622 27.678113 20.660302
## [64] 17.945242 27.586245 31.006722 18.883230 24.662088 25.849537 17.029625
## [71] 17.255490 18.309076 21.771059 8.850397 23.037457 19.099747 20.712154
## [78] 17.574216 26.476857 30.326047 30.639892 23.031025 19.598532 26.831297
## [85] 27.182925 19.128179 23.043928 21.295316 16.445118 30.799429 22.680780
## [92] 14.439768 21.698185 16.909359 28.720221 19.338222 19.283668 24.427489
## [99] 23.521608 30.154735 23.172312 22.313385 21.353226 23.590980 8.536840
## [106] 24.646673 23.076392 24.096677 22.629943 26.535825 12.439151 18.647854
## [113] 24.773420 31.097951 22.398756 22.883207 14.428842 15.655816 14.550404
## [120] 23.102923 26.841792 25.035425 28.395900 20.398345 24.639253 29.968666
## [127] 15.559437 19.984968 23.948936 17.529481 21.023694 20.034654 34.754367
## [134] 26.402327 21.238236 21.987409 14.831805 29.709599 24.660696 18.190234
## [141] 23.933253 22.780837 14.143874 24.606953 17.409134 22.056133 22.918149
## [148] 21.292872 15.695770 17.850405 23.246530 19.989352 30.247344 13.338843
## [155] 29.338006 23.068920 26.279073 17.643252 15.319054 16.910815 22.486020
## [162] 32.644755 21.646968 26.186679 20.379631 19.100401 26.670818 25.963521
## [169] 19.532585 24.511634 16.630501 19.208109 26.106679 24.274778 22.886239
## [176] 23.281333 26.136013 17.405639 15.297149 19.311486 19.345933 17.274980
## [183] 19.487363 20.875580 23.412399 20.453108 26.296725 22.379133 21.638840
## [190] 18.969769 20.812941 20.734678 22.405083 19.614253 23.030022 23.414688
## [197] 21.319170 26.709908 20.683153 32.045845 26.398390 19.975308 20.277140
## [204] 25.995826 25.167097 20.393108 21.638608 30.941371 32.690196 25.599939
## [211] 20.164505 16.164837 26.382373 13.417886 17.302122 27.808092 22.360328
## [218] 23.187862 20.761771 18.737034 23.842472 29.441578 26.041518 32.415377
## [225] 21.769106 20.451204 34.688940 23.405212 23.304027 23.770183 28.149672
## [232] 21.096531 30.276991 18.592079 29.520995 26.965684 17.794927 21.691161
## [239] 25.814822 17.066922 27.056328 15.578823 20.997653 18.735860 25.986853
## [246] 20.030516 10.313346 25.772127 26.729288 19.893636 19.225869 15.775628
## [253] 20.085726 24.325385 7.530148 15.491612 11.893855 11.432904 32.816053
## [260] 17.513009 17.232583 23.101267 27.433700 19.877154 17.214477 20.881844
## [267] 15.845974 30.060359 20.618824 13.764043 26.116151 19.148865 27.463203
## [274] 20.657739 21.105859 20.475611 22.153065 14.311917 16.883267 29.574003
## [281] 30.669654 28.166801 16.479501 30.789881 21.559212 25.162837 19.614768
## [288] 23.284124 13.830230 19.593676 27.630178 20.688705 28.001314 14.702296
## [295] 27.141221 24.957388 20.512948 14.113858 26.494781 23.374542 11.453836
## [302] 23.527704 19.421047 18.709197 26.221022 18.868644 17.374772 25.117308
## [309] 12.194836 18.607335 22.972916 19.733196 17.333940 21.718328 27.022687
## [316] 23.966969 21.279434 28.648465 27.980911 25.588034 30.135920 24.388457
## [323] 24.741219 16.235548 26.853350 30.141355 37.034139 23.885693 25.096541
## [330] 25.448287 23.609092 24.970892 24.431860 20.005020 19.004746 18.800766
## [337] 12.684321 23.375571 24.693360 25.717833 18.025998 27.066586 19.254281
## [344] 22.667097 10.393697 17.432945 20.737361 25.680239 30.554683 21.513875
```

```
w1 <- length(z1)
```

```
w1
```

```
## [1] 350
```

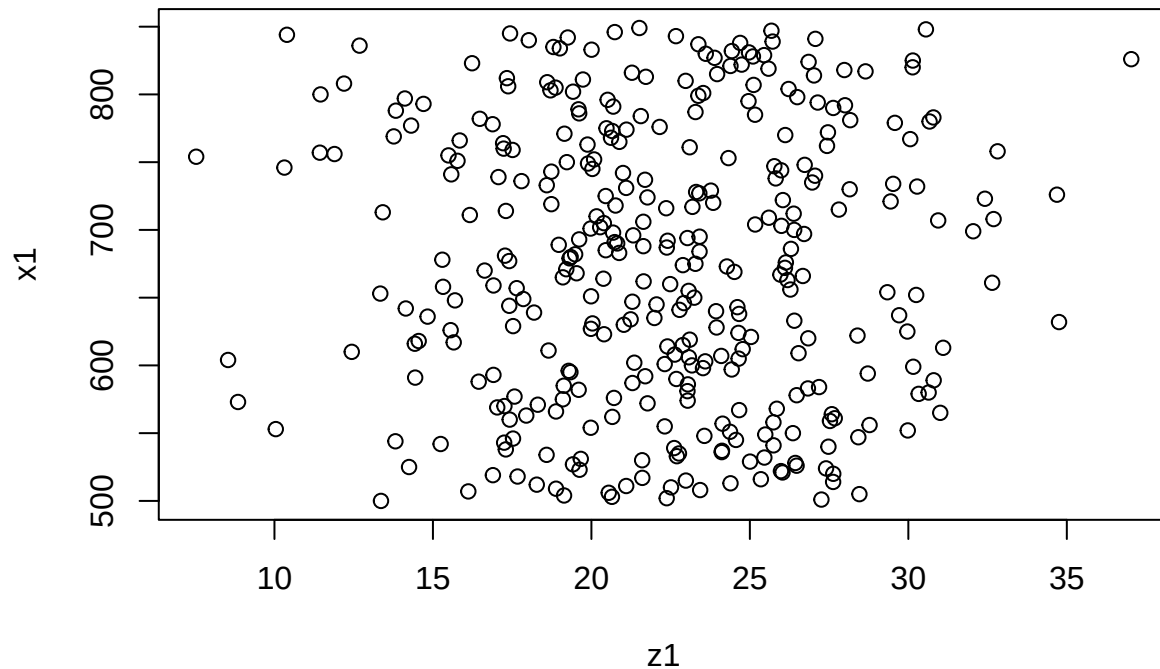
```
x1 <- 500:849
```

```
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
```

```
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

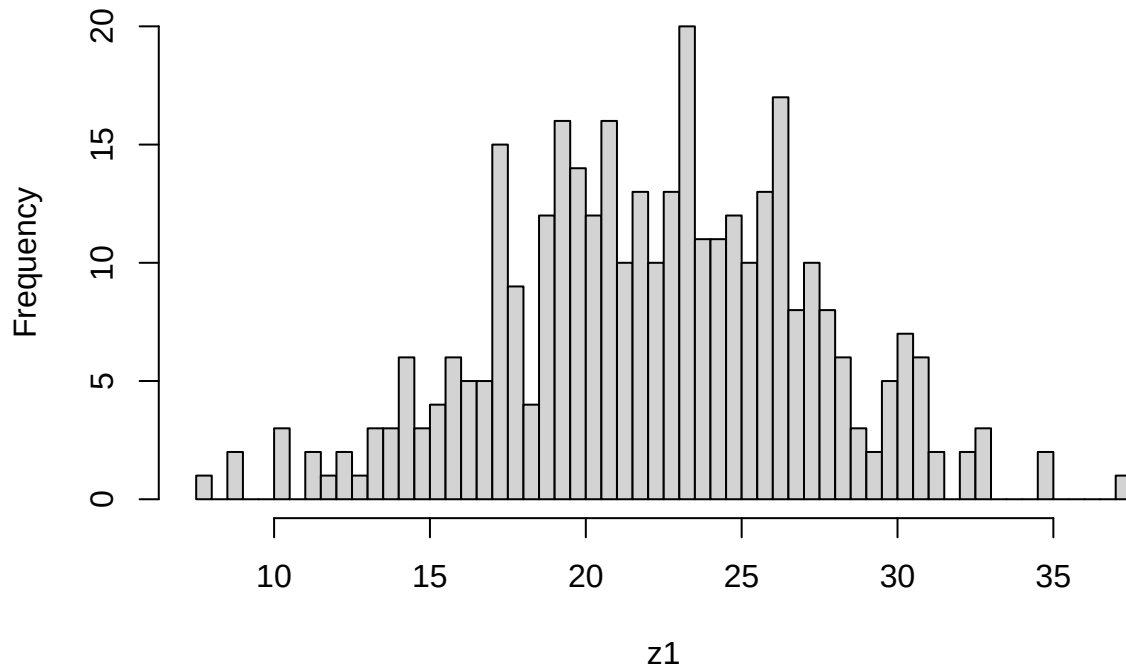
```
plot(z1, x1)
```



El comando `hist()` genera un histograma.

```
hist(z1, main = "Histograma de edades", breaks = 60)
```

## Histograma de edades



El comando `density()` calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :  
## extra argument 'type' will be disregarded
```

```
##
```

```
## Call:
```

```
## density.default(x = z1, type = "b")
```

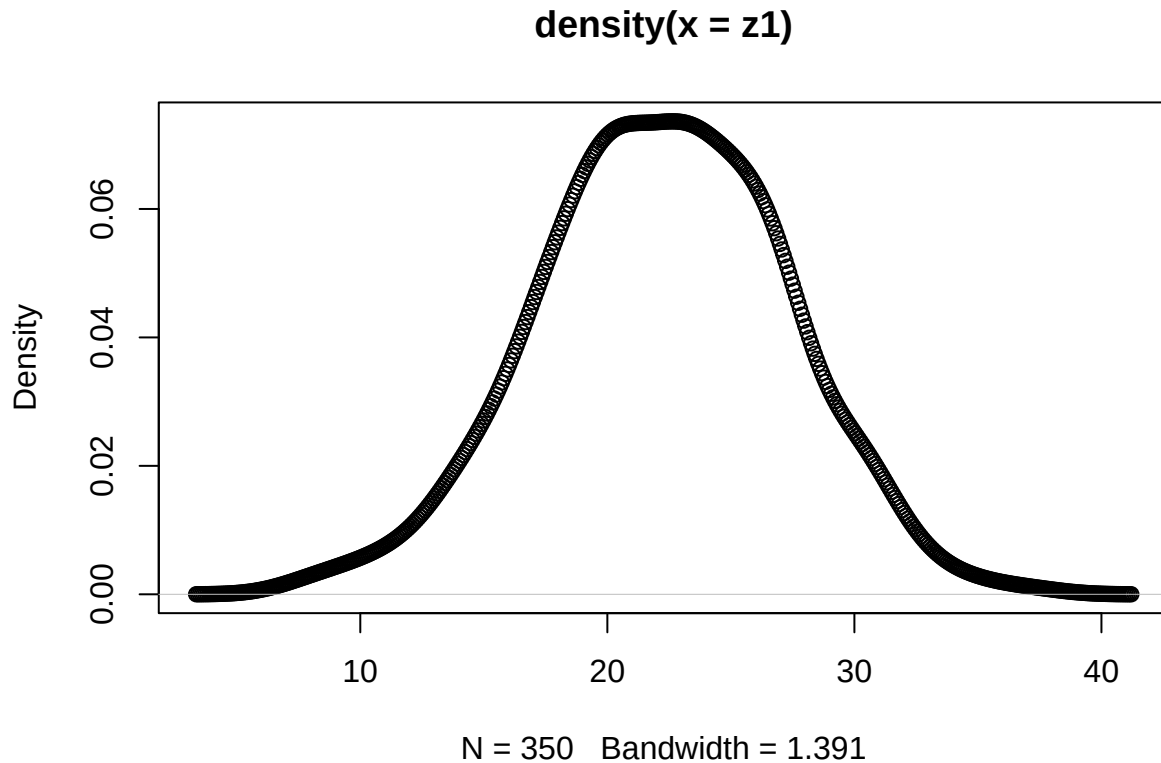
```
##
```

```
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.391
```

```
##
```

```
##      x              y  
## Min.   : 3.357   Min.   :9.185e-06  
## 1st Qu.:12.819   1st Qu.:2.106e-03  
## Median :22.282   Median :1.411e-02  
## Mean   :22.282   Mean    :2.637e-02  
## 3rd Qu.:31.745   3rd Qu.:5.259e-02  
## Max.   :41.208   Max.    :7.364e-02
```

```
plot(density(z1),type="b")
```



## Ejercicio 1: Designación

**Consigna:** Las ultimas 3 cifras del DNI son 858, crear una variable que tenga ese número

```
DNI <- 858
```

**Consigna:** Crear un vector del 1 al 858.

```
secuencia_dni <- 1:858
secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810
## [811] 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828
## [829] 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846
## [847] 847 848 849 850 851 852 853 854 855 856 857 858
```

---

## Ejercicio 2: Uso de for

**Consigna:** Calcular la suma de todos los valores del vector *secuencia\_dni* usando un for:

```
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total

## [1] 368511
```

---

## Ejercicio 3: Python

**Consigna:** Repetir el ejercicio anterior pero en Python

```
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (858)
dni = 858
```

```

# Consigna: Crear un vector (lista) del 1 al 858
# En Python, range(1, 804) genera números del 1 al 858
secuencia_dni = list(range(1, dni + 1))

# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")

## La suma total es: 368511

```

---

## Ejercicio 4: Medición de tiempo (sys.time)

**Consigna:** ¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando `sys.time`

```

inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total

## [1] 3.68082e+19

final <- Sys.time()
final-inicio

## Time difference of 2.427159 mins

```

---

## Ejercicio 5: 2da medición de tiempo (tictoc)

**Consigna:** Aplicar la consigna anterior pero aprendiendo a usar el comando `tictoc`

```

library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}

# Fin del bloque a medir

```



```

#Se "detiene" el cronómetro.

#Esto imprimirá el tiempo transcurrido en pantalla.
toc()

## Tiempo de ejecución: 0.004 sec elapsed
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total

## [1] 368511

```

---

## Ejercicio 6: Secuencia

**Consigna:** Generar secuencia de 2 en 2 hasta 50000

```

inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for

```

Secuencia con función de R:

```

inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for

```

```
## Time difference of 0.005922318 secs
```

```
tiempo_seq
```

```
## Time difference of 0.001852989 secs
```

---

## Ejercicio 7: Serie de Fibonacci

**Consigna:** Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```

# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]

```

```

    fibonacci <- c(fibonacci, nuevo)
    iteraciones <- iteraciones + 1
}

fibonacci

## [1]      0      1      1      2      3      5      8     13     21
## [10]     34     55     89    144    233    377    610    987   1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269

iteraciones

## [1] 32

```

---

## Ejercicio 8: Definición matemática recurrente

**Consigna:** ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```

f0<-0 #Valores iniciales
f1<-1
f2<-0
i<-0 #Número de iteraciones inicial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2

## [1] 1346269

i

## [1] 30

```

---

## Ejercicio 9: Método Burbuja (Bubble Sort)

**Consigna:** Compara la performance de ordenación del método burbuja vs el método sort de R.

```

x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
}

```

```

    }
  }
}
return(x)
}

```

Comparación:

```

system.time({
  res1 <- burbuja(x)
})

```

```

##      user  system elapsed
## 21.656   0.004  21.779

```

```

system.time({
  res2 <- sort(x)
})

```

```

##      user  system elapsed
##      0      0      0

```

---

## Ejercicio 10: Penitencia de Newton

En archivo aparte

---

## Análisis de Rendimiento: Ordenamiento y Agrupación

```

library(microbenchmark)
library(ggplot2)

# Definimos la función del método burbuja (del apunte original)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generamos una muestra aleatoria moderada para la prueba
set.seed(123)
muestra <- sample(1:1000, 400, replace = TRUE)

# Ejecutamos el benchmarking
mbm_orden <- microbenchmark(

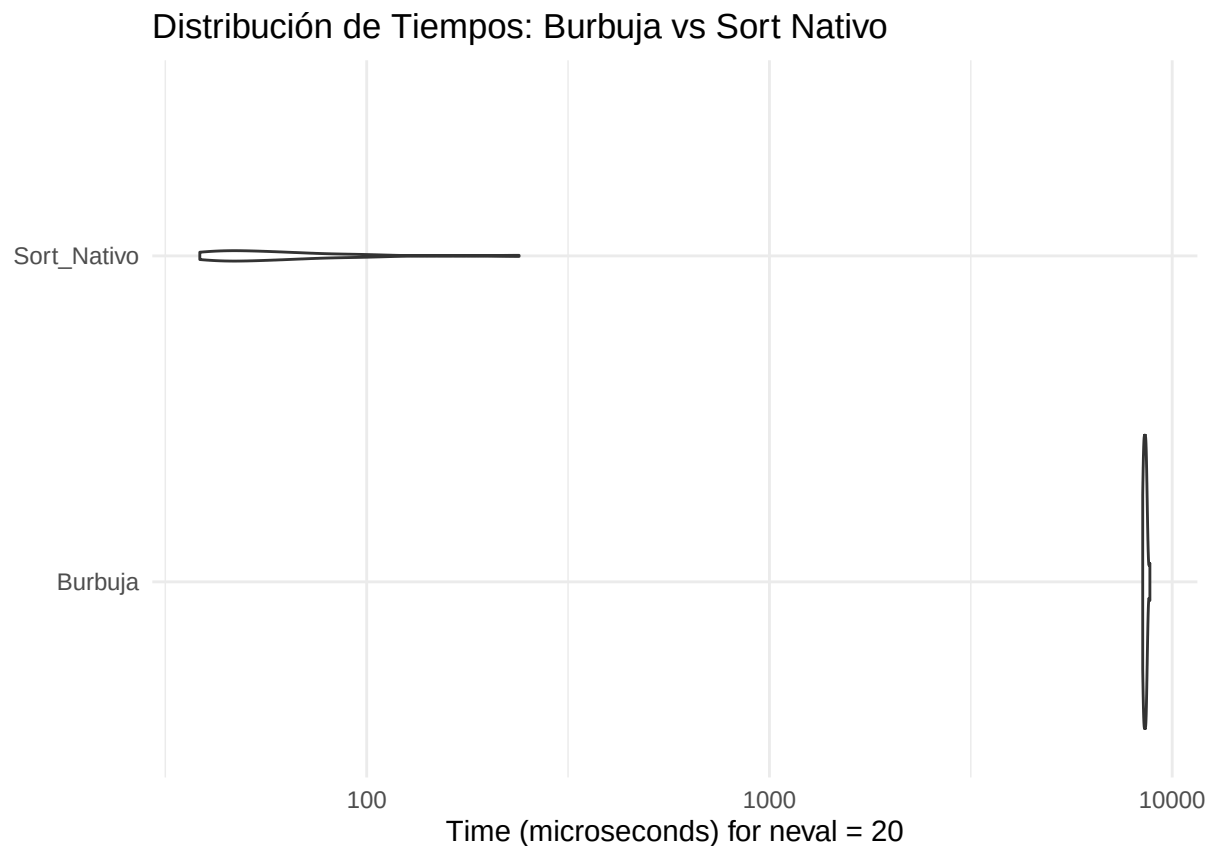
```

```

Burbuja = burbuja(muestra),
Sort_Nativo = sort(muestra),
times = 20 # Repetimos 20 veces para tener una buena distribución
)

# Visualizamos con gráfico de violín
autoplot(mbm_orden) +
  ggtitle("Distribución de Tiempos: Burbuja vs Sort Nativo") +
  theme_minimal()

```



```

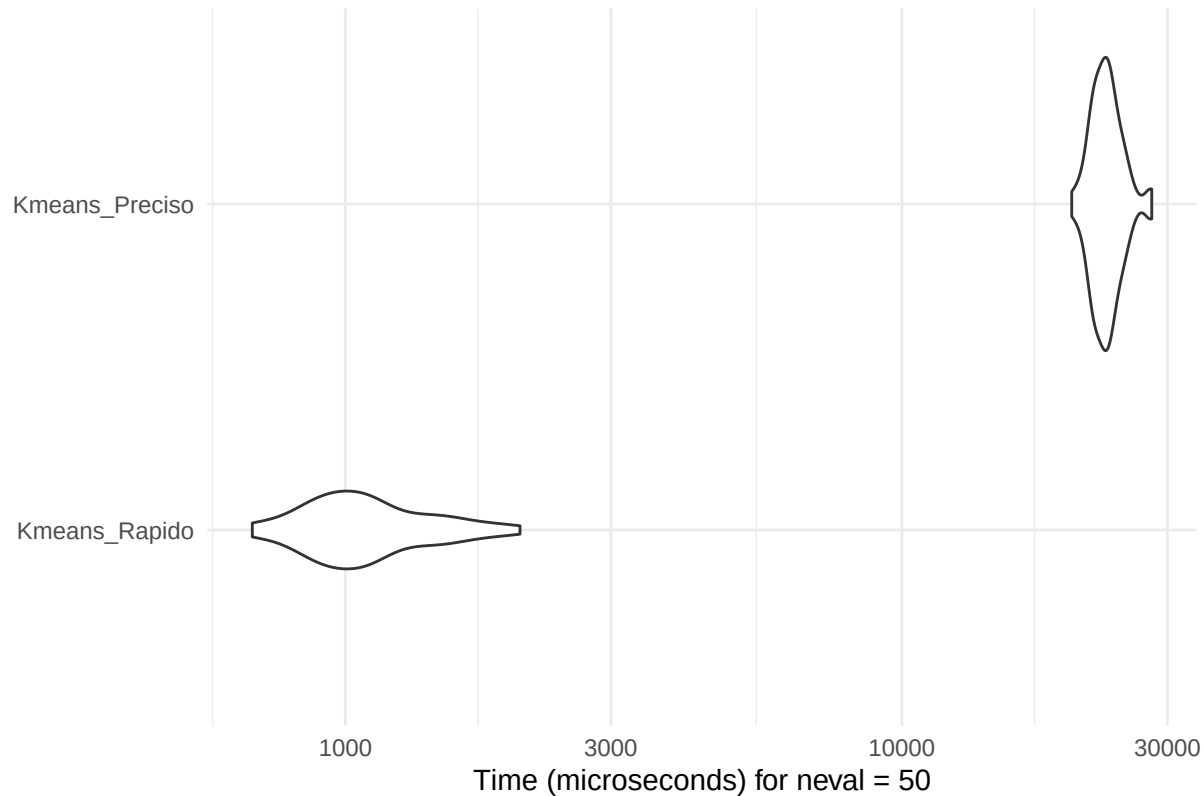
# Generamos una matriz de datos simulada (ej: variables de mantenimiento industrial)
set.seed(42)
datos_industriales <- matrix(rnorm(5000), ncol = 5)

# Benchmarking del algoritmo K-Means
mbm_kmeans <- microbenchmark(
  Kmeans_Rapido = kmeans(datos_industriales, centers = 4, nstart = 1),
  Kmeans_Preciso = kmeans(datos_industriales, centers = 4, nstart = 25),
  times = 50
)

# Visualizamos con gráfico de violín
autoplot(mbm_kmeans) +
  ggtitle("Performance de K-Means: Impacto del parámetro nstart") +
  theme_minimal()

```

## Performance de K-Means: Impacto del parámetro nstart



```
# Librerías necesarias
install.packages("ggplot2")

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
## (as 'lib' is unspecified)

install.packages("cluster")

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
## (as 'lib' is unspecified)

library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)
```

```
# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)
```

```
## El coeficiente de silueta promedio es: 0.6808356
```

```
# 4. Visualización con Gráfico de Violín
```

```
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()
```

